

En este proyecto se tiene como objetivo desarrollar algoritmos, tales como Bubble, Selection e Insertion, para organizar y procesar datos. A través de este trabajo, se busca reforzar e implementar los conceptos esenciales aprendidos durante la cursada y afrontar los problemas de manera lógica, ordenada y eficiente.

Los algoritmos de ordenamiento propuestos cumplen con la misma finalidad, con distintos medios, para ejecutarlos se le asignó uno a cada miembro del grupo.

Para el desarrollo de los algoritmos se tenía dos funciones obligatorias:

init(vals)

La función se ejecuta al comenzar o cada que se remezcla la lista de valores. En esta etapa se realiza una copia de la lista original para evitar modificar el arreglo recibido por parámetro (**items = list(vals)**), se guarda la longitud de la lista en una variable **n**, y además se inicializan las variables de control que usará el algoritmo, como por ejemplo **j** para que luego pueda ser utilizada dentro de la función **step()**.

step()

Por otra parte, esta función representa un “micro paso” del algoritmo. Cada vez que es llamada, el algoritmo avanza un único paso pequeño. Al finalizar cada llamada, **step()** debe devolver un diccionario que indica qué ocurrió en ese paso: los índices **a** y **b** que se están comparando, un valor booleano **swap** que es **True** solamente si en ese paso se realizó un intercambio entre esos dos elementos, y un valor **done** que se vuelve **True** cuando el algoritmo terminó completamente de ordenar.

Algoritmos

El método burbuja (**Bubble Sort**) consiste en comparar a los elementos adyacentes de la lista e intercambiarlos si se encuentran en una posición incorrecta. De este modo, los “ítems” de mayor valor se desplazan hacia el final, generando un orden de menor a mayor.

Debido a las características del visualizador, los dos bucles anidados clásicos del algoritmo, tuvieron que ser reemplazados por variables de estado **i** y **j** que actúan como índice externo e índice interno. De este modo, se puede avanzar un paso a la vez mediante la función **step()**.

En cada ejecución, se comparan **items[a]** e **items[b]** (siendo $a = j$ y $b = j + 1$), actualiza los contadores y devuelve la información del paso.

Al momento de implementarlo, surgieron algunas dificultades con el manejo de los índices por la comparación de los elementos adyacentes y con definir cuando debía terminar el algoritmo. Además, fue necesario asegurar que los elementos se intercambiarán de manera

correcta en cada pasada. Esto permitió entender cómo el algoritmo reduce las comparaciones a medida que los valores más grandes son desplazados hacia el final de la lista.

El método de Selección (**Selection Sort**) recorre la lista para tomar el valor más pequeño de la parte no ordenada, cuando lo encuentra lo intercambia con el elemento que se encuentra al inicio de la sección, organizando el algoritmo de menor a mayor repitiendo este proceso.

Teniendo en cuenta las características del visualizador en este caso también se tuvo que reemplazar los bucles anidados por variables de estado, como **i**, **j** y **min_idx**. En este caso **i** es el inicio de la sección no ordenada y **j** recorre esa parte para encontrar el elemento mínimo mientras que **min_idx** guarda el valor más pequeño que poseen hasta el momento. De este modo, el algoritmo avanza un paso a la vez.

Para poder ejecutar el algoritmo de ordenamiento, se lo dividió en dos fases internas: “**buscar**” y “**swap**”. En la primera, se compara `items[j]` con `items[min_idx]` para confirmar si se encontró un nuevo valor mínimo, haciendo que **j** recorra la parte no ordenada de la lista. Mientras que en la segunda fase, “**swap**”, se genera el intercambio cuando **j** encuentra un elemento menor al actual mínimo, provocando un reemplazo entre el valor encontrado y el ubicado en **i**.

A la hora de ensamblar el algoritmo, surgió la dificultad de comprender que la función **step()** debía generar un paso a la vez. Al comienzo se intentó que el código repitiera el proceso internamente, siendo una idea errada, debido a que el visualizador tiene la funcionalidad de repetir la función de paso automáticamente. La confusión se pudo disipar fácilmente viendo material de referencia. Además, hubo inconvenientes para ubicar las fases “**buscar**” y “**swap**” de forma que el algoritmo pudiera funcionar de forma correcta y fluida en el orden deseado.

El método de Inserción (**Insertion sort**) arma la lista ordenada de forma progresiva. En cada paso, toma un elemento de la parte no ordenada y lo desplaza hacia la izquierda hasta encontrar su lugar correspondiente en la lista. Este proceso se repite para cada elemento, recorriendo toda la lista. Insertando en la posición correcta obtiene un orden de menor a mayor.

La implementación de Insertion se realizó en forma de micro paso para permitir la visualización detallada del algoritmo. Se mantiene una variable **i** que marca el elemento a insertar y un cursor **j** que se desplaza hacia la izquierda comparando e intercambiando cuando el elemento anterior es mayor. Cada llamada a **step()** realiza una comparación o

un intercambio, cuando ya no es posible desplazar más a la izquierda, se fija su posición haciendo que **i** avance al siguiente elemento.

La dificultad con este último método no fue muy distinta a los problemas que surgieron en el resto del proyecto. El visualizador propuesto limitaba el uso de bucles, lo que generó un desafío extra a la hora de implementar el algoritmo. Aun así, pudimos adaptarnos a esas restricciones y encontrar una solución funcional. También tuvimos inconvenientes para trabajar en Github, a la hora de subir los cambios y avances de cada integrante, pero una vez que un miembro supo cómo manejarse en la plataforma, el resto pudo adaptarse.